The Re-verification Cost of Evolving Distributed Protocols: A Pilot Study on MongoDB's TLA+ History

AMAYENE SEKE Louis Quentin
Douala, Cameroon
amayenelouisgalois@gmail.com

Technical report, version 3, October 2026

Abstract

Distributed protocol specifications keep changing, and each edit can invalidate earlier verification results. We measure re-verification cost with TLC on two real commits from MongoDB's public TLA+ specifications, under a controlled protocol with machine-checked integrity for each commit pair. Adding two safety invariants, with the protocol unchanged, was associated with a 57% rise in checking time on an identical state space; with one liveness property added as well, a single run capped at 600 s did not terminate. A 12-line change to a vote rule removed 34% of the symmetry-reduced state space in a 3-server model. The edit only strengthens a guard, so by refinement the checked safety properties, which do not refer to `ENABLED`, carry over; canonicalizing states under server symmetry confirms that every child state was already reachable in the parent. Full re-exploration nevertheless revisited 2.4 million states. A raw textual comparison had suggested 371,889 new states, all of them artifacts of symmetry. Since incremental model checking is known to be unbounded in general, we discuss what these observations imply for bounding re-verification cost per class of specification-level edit, and for a benchmark of versioned protocol histories.

1 Motivation and research gap

Neuro-symbolic invariant inference has made substantial recent progress for distributed protocols. The preprint IC3Syn (Cao, Wu, Yao, Wei, Chen, Ma [2]) combines a symbolic IC3 controller with an LLM over TLA+ specifications. Its abstract reports candidate invariants for all 29 protocols in its evaluation, including MongoDB's MongoLoglessDynamicRaft, with the invariants found on finite instances shown in TLAPS to be inductive for the unbounded protocol. A paper by the same six authors, "I3DP: Neuro-symbolic Inductive Invariant Inference for Distributed Protocols," is published in the SOSP 2026 proceedings [1]; its abstract reports the same results (29 protocols, including MongoLoglessDynamicRaft, with TLAPS proofs of inductiveness). We have read the abstracts of both, not the full papers. Such systems work inside a loop that repeatedly checks, refines, and re-checks candidate invariants. In engineering practice, protocols also keep changing, and each edit can invalidate earlier verification results.

Practitioners treat re-verification after a change as a criterion for choosing a method. W. Schultz (MongoDB Research) writes on his personal blog that, compared with fully automated proofs, he often still finds "finite-state model checking over adequately large state spaces" to be "a better cost-benefit tradeoff," one that "makes things much easier to automate and re-verify upon modification" [3]. This study measures what that re-verification costs when the model checker is not incremental.

Research gap (A3). We label the gap below A3, as in the author's research plan. Incremental model checking has a long history. Sokolsky and Smolka [4] give an incremental algorithm for the alternation-free modal μ-calculus. Its cost is bounded in terms of Δ, the set of inserted or deleted transitions of the explicit labeled transition system: linear in the size of the LTS in the worst case, and linear in Δ in the best case. They also show, by reduction from single-source reachability, that model checking is an *unbounded*

problem in the sense of Ramalingam and Reps: the running time of an incremental update cannot, in general, be expressed solely in terms of Δ. Swamy, Brayton and Singhal [18] update the reachable set of a finite state machine after a design change instead of recomputing it. Kähler and Wilke [19] study the dynamic complexity of LTL model checking. Lauterburg et al. [20] make explicit-state exploration incremental and evaluate it on 24 program evolutions and exploration changes. Regression model checking [5] and the reuse of verification results [6, 21] address the problem empirically, the latter on 1,119 revisions of 62 Linux device drivers. For hardware, Chockler et al. [22] reuse IC3 inductive invariants across design revisions by keeping the largest subset of old clauses that remains inductive under the new transition relation; the SAT calls of that reuse step are bounded by the number of old clauses discarded, but the total cost is not bounded. IPDR [23] reuses PDR lemmas across instances whose constraints are tightened or relaxed. Proof-based tools already avoid part of the work: the TLAPS proof manager remembers which obligations were proved and does not send them to a back-end prover again [15], Dafny caches verification results at a fine grain [16], and inductive proof decomposition [17] splits a protocol's safety proof by action. These reuse proof results; they do not bound the cost of re-checking a model.

Given the unboundedness result, no useful bound can be stated for arbitrary edits; a bound can only hold for restricted classes of edits or properties. To the best of our knowledge (literature survey through October 2026), no published work identifies, for explicit-state model checking of distributed protocol specifications, classes of *specification-level* edits (diffs to the protocol text) together with bounds on re-verification cost, in verifier invocations or any other measure, nor does any do so for the invariant-inference loops of neuro-symbolic systems. Existing incremental algorithms take Δ as input. Result 2 below illustrates why this is hard to meet in practice: Δ is not evident from the textual diff.

This pilot study is an initial empirical characterization of that cost for distributed protocol specifications. It is meant as groundwork for a benchmark (DistInv-Evolve) and a theory.

2 Method

We mine real commits from MongoDB's public TLA+ specifications (`src/mongo/db/repl/tla_plus` in `mongodb/mongo` at the studied commits; the directory has since moved) and measure verification cost with TLC 2.19 under a controlled protocol:

- each datapoint is a (parent, child) commit pair;
- the complete `tla_plus` directory is materialized at both commits;
- configurations are identical except for documented single-variable reductions;
- integrity is machine-checked for each change type:
  - protocol-logic changes (Result 2): configuration diffs must be empty, and only the target `.tla` file may differ between the two runs;
  - obligation-only changes (Result 1): the `.cfg` file may differ only in `INVARIANT` or `PROPERTY` lines and comments, and the `.tla` file only in comments; scripts that our runs do not use (`model-check.sh`) are excluded.

Environment. TLC2 2.19 (rev 5a47802, 2024-08-08; release tag v1.7.4 [14]), invoked as `java -XX:+UseParallelGC -jar tla2tools.jar -workers 1`; 1737 MB heap as reported by TLC; MSBDiskFPSet / DiskStateQueue; Oracle JDK 21.0.3; Ubuntu, kernel 6.8.0-138; 4 cores, 7.6 GB RAM.

At a fixed worker count, state counts are deterministic. They were identical across independent reruns, including reruns with different fingerprint functions. Result 1 times are TLC-reported means of three alternated runs with an identical JVM invocation; Result 2 times come from two runs in which parent and child were checked concurrently; the 4-server time is a single run. Deadlock checking was left at TLC's default. Every run whose state counts or times we report ended with "Model checking completed. No error has been found." The only runs that did not complete are the 4-server runs and the capped liveness run, as stated in the text.

3 Result 1: obligation-only change

RaftMongo, commit `348880238d`, "SERVER-45853 Enable RaftMongo.tla invariants", A. Jesse Jiryu Davis, 2020-01-30.

The commit adds verification obligations without changing the protocol. In `MCRaftMongo.cfg` it adds `INVARIANT NeverRollbackCommitted`, `INVARIANT NeverRollbackBeforeCommitPoint`, and `PROPERTY CommitPointEventuallyPropagates`. Its edits to `RaftMongo.tla` touch only comments. The commit also edits `model-check.sh` (worker count, ParallelGC); those edits do not affect our runs, which use our own fixed TLC invocation.

The parent configuration already checks `INVARIANT NoTwoPrimariesInSameTerm`; it uses `Server = {1, 2, 3}` and no `SYMMETRY`, so the warning about symmetry and liveness does not apply here. The child configuration notes that the two added invariants can be violated, "although it's not ultimately a safety issue" (SERVER-39626), with at least 5 servers, 3 terms and oplogs of length 4 or more; our instance is below these limits.

The middle row of Table 1 does not correspond to any commit. It is an intermediate configuration we built ourselves: the parent configuration plus the two `INVARIANT` lines, without the `PROPERTY` line.

Obligations	States (generated / distinct)	Time (TLC, mean of 3)
Parent configuration (baseline)	5,398,516 / 371,368	105 s (104–106)
+ 2 safety invariants (constructed)	5,398,516 / 371,368	165 s (162–169), +57%
+ 2 invariants + 1 liveness (child)	not terminated	$>$ 600 s (single run)

Table 1: Result 1: adding verification obligations without changing the protocol.

In this instance, the two safety invariants were associated with a 57% rise in checking time (mean of three alternated runs; the six raw times are in the accompanying logs) while exploration stayed identical. We attribute this to per-state invariant evaluation but did not isolate it. With the liveness property added on top of the two invariants, a single run capped at 600 s did not terminate. This is a censored observation: we did not run the parent with the liveness property alone, nor an uncapped run, so we draw no conclusion on the cost of liveness checking beyond "more than 600 s in our configuration." One observation does not establish a general cost-class separation.

4 Result 2: protocol-logic change

MongoReplReconfig, commit `dd27888039`, "SERVER-47430 Update TLA+ to only vote for candidate with same config version and term as self", Siyuan Zhou, authored 2020-04-02, committed 2020-04-17; cherry-pick of `6e182b5c907f` (same author date, committed 2020-04-09).

This developer commit restricts the vote rule used during reconfiguration. It matches the server code of the time: at both commits, `processReplSetRequestVotes` in `topology_coordinator.cpp` refuses a vote whenever the candidate's config differs from the voter's. In `CanVoteFor`, the predicate `IsNewerConfig` (configTerm strictly greater, or configTerm equal and configVersion $\geq$) is replaced by `HasSameConfig` (configTerm equal and configVersion equal). The diff has 12 changed lines (5 added, 7 removed) and touches one predicate and one call site.

With the shipped configuration (4 servers, `SYMMETRY ServerSymmetry`, `INVARIANT ElectionSafety`, `PROPERTY NeverRollbackCommitted`), exhaustive verification had not converged after 43 minutes in a single run (one worker, default heap); the frontier queue was growing by roughly 80,000 states per minute. We did not pursue this configuration and ran a documented single-variable reduction: 3 servers, everything else identical (Table 2). The configuration of this specification on MongoDB's current master (checked 2026-10-29) also uses 3 servers.

Version	States generated	Distinct states	Time (two runs)
Parent (`IsNewerConfig`)	63,107,985	3,680,146	29 min 54 s; 30 min 15 s
Child (`HasSameConfig`)	41,496,087	2,413,165	19 min 23 s; 20 min 51 s

Table 2: Result 2: 3-server reduction of MongoReplReconfig before and after SERVER-47430.

Each version was checked twice. In both runs, parent and child were checked concurrently on the same 4-core machine, one TLC worker each, so wall-clock times include contention and are indicative only. State counts were identical across runs.

A syntactically minimal change removed 34.4% of the reachable states counted up to symmetry (orbits); the ratio for the unreduced state space may differ. Generated states fell by a similar fraction (34.2%), so the average number of successors per distinct state is almost unchanged (17.15 in the parent, 17.20 in the child). The direction of the effect was expected, since a stricter rule prunes; its magnitude is not evident from the diff.

The shipped configuration uses symmetry reduction together with a temporal `PROPERTY`, and the configuration file itself warns that symmetry "may invalidate liveness checking in certain cases." We report state counts only and draw no liveness conclusions from these runs.

5 State-space overlap between parent and child

We dumped both reachable state sets (from the second run) and compared them after canonicalizing each state under server symmetry: for each state, we took the minimal serialization over all 3! permutations of the server identifiers, re-sorting sets and function domains after renaming. The script parses each value and renames only whole model-value tokens, never substrings, and re-sorts at every nesting level (for example the server sets inside `config`). Without permutation, this procedure reproduces the raw textual intersection (2,041,276); with or without permutation, the number of distinct forms equals the TLC distinct count of each run.

|parent|	3,680,146
|child|	2,413,165
|intersection (canonical)|	2,413,165

- Every child state, up to symmetry, was already reachable in the parent.
- 65.6% of the parent's state space survives the change.
- The change introduces no new state.

The raw textual intersection had suggested 371,889 new states. All of them were the same states written under a different permutation of the servers.

This inclusion is what the edit predicts, so it mainly validates the canonicalization; it is not an independent finding. The argument, with its assumptions:

- `HasSameConfig` implies `IsNewerConfig`. The diff deletes the definition of `IsNewerConfig`, so the module could not load if it had any other use. Its replacement occurs only in `CanVoteFor`, which occurs only in `BecomeLeader`, as a conjunct of the guard under $\exists$ `voteQuorum`, $\forall$ `v` $\in$ `voteQuorum`: never under a negation, an `IF`, a `CHOOSE`, or an expression assigned to a primed variable. Hence every step of the child's next-state relation is a step of the parent's.
- `Init`, `StateConstraint` and `ServerSymmetry` (`Permutations(Server)`) are unchanged. The only `CHOOSE` in the module ranges over integers (`Min`, `Max`), so server symmetry is not broken by it.

- `Spec` also contains `Liveness`, a conjunction of six weak-fairness conditions (on `BecomeLeaderAction`, `SendConfigAction`, `UpdateTermsAction`, `GetEntryAction`, `RollbackEntriesAction` and `CommitEntryAction`), so the child's behaviours are not a subset of the parent's. This does not affect safety: all six are on sub-actions of `Next`, so the safety closure of each `Spec` is `Init` $\wedge$ `[][Next]_vars`.
- The conclusions below concern the finite model under `StateConstraint`; the transfer of an inductive invariant also holds for the unbounded protocol.

Therefore every state predicate that does not refer to `ENABLED`, and every action property of the form `[][P]_vars`, that holds in the parent holds in the child. This covers both checked properties: the invariant `ElectionSafety` and the action property `NeverRollbackCommitted`. An inductive invariant of the parent that does not refer to `ENABLED` remains inductive for the child.

Mechanical check. We also checked the step inclusion with TLC. A wrapper module extends the child's model-checking module, instantiates the parent module (renamed `MongoReplReconfigParent`) with the same constants and variables, and checks `PROPERTY Parent!Init` and `PROPERTY [][Parent!Next]_vars` on the child, with the 3-server configuration of Table 2. TLC reported "Model checking completed. No error has been found" after 41,496,087 generated and 2,413,165 distinct states, the same counts as our child runs (2 workers, a different machine, 39 min). As a control against a vacuous check, the converse property (the parent refines the child) is violated after three states: a re-election in which the candidate's `configTerm` differs from its voter's. Both modules are included in `pilot/refinement_check/` of our repository.

It does *not* cover predicates that refer to `ENABLED` of the affected action. The module itself defines one, `ElectableNodeExists` $\triangleq \exists s \in$ `Server : ENABLED BecomeLeader(s)`, which the shipped configuration does not check. Strengthening the vote guard shrinks `ENABLED BecomeLeader(s)`, so such a predicate can hold in the parent and fail in the child. Liveness properties are not covered either, nor is deadlock freedom, which TLC checks by default and which is a statement about `ENABLED Next`; it held in all our runs.

Full re-exploration of the child nevertheless revisited 2.4 million states, all already explored, to re-establish two properties that the argument above already guarantees.

Caveat: state overlap is not transition reuse. A shared state does not imply a preserved transition: the edited predicate can remove successors from states where a vote is possible. State overlap is a precondition for reuse, not a measure of it: it indicates how much an incremental method could hope to reuse, and it is neither a guarantee nor an algorithm. In the terms of [4], the quantity an incremental checker would need is Δ, the set of changed transitions, and the textual diff does not reveal its size.

6 Implications for the research gap (A3)

Three observations, each from a single case:

(i) In Result 1, cost depended strongly on which obligations were added (two safety invariants, then a liveness property).

(ii) A local edit reshaped the state space substantially, by a magnitude not evident from the diff (Result 2).

(iii) The edit strengthens a guard, so re-checking the two shipped safety properties was redundant; the canonical comparison confirms that no new state appeared (Section 5).

Incremental model checking is unbounded in general [4], and existing incremental algorithms take the changed transitions (Δ) as input. A3 therefore asks for bounds per class of specification-level edit and class of property, not for a general bound. To our knowledge, no model predicts re-verification cost

from a specification-level edit of a distributed protocol. Measured verification costs over real program revisions exist [21], as do measurements over program evolutions [20]; we know of no such dataset for distributed protocol specifications, which DistInv-Evolve aims to provide.

Guard strengthening is one such class: for safety properties that do not refer to `ENABLED` of the affected actions, re-verification is unnecessary, and incremental PDR exploits tightened constraints in a related way [23]. TLC is not incremental and re-explores. The class is narrower than "guard strengthening" alone: the same edit can falsify an `ENABLED`-based predicate or a liveness property. A benchmark should label each edit by class and each property by kind, so that measured costs can be compared within a class.

The studied guard did not stay strengthened in the implementation. On MongoDB's current master (checked 2026-10-29), `processReplSetRequestVotes` refuses a vote only when the candidate's config is older than the voter's, and the replication README states that a voter with config (v,t) votes only if the candidate's config $(v_c,t_c) \geq (v,t)$, which is the parent's `IsNewerConfig` rule. The TLA+ specification on master still uses `HasSameConfig`. The server rule was relaxed under SERVER-57262 (commit `e7937256045a`, 2021-08-02, which replaces `!=` by `<` in the vote check). The ticket gives the reason: requiring the same config "causes a liveness issue where if a primary is stuck in catchup/drain mode with a lower config, catchup takeover cannot bail it out", so that "the whole replica set will hang because there will be no writable primary" [13]. This is precisely what the argument of Section 5 does not exclude: the strengthened guard preserved safety and harmed liveness. The corresponding specification update, SERVER-58788 ("Update TLA+ to allow voting for candidate with higher config", created 2021-07-23), was still open on 2026-10-29 [13]. It would be an edit of the opposite class, a guard weakening, which can add states and for which safety must be re-checked.

7 Related work

Incremental and regression model checking. Sokolsky and Smolka [4] give an early incremental model-checking algorithm, whose cost is linear in the size of the LTS in the worst case and in $|\Delta|$ in the best case; they show the problem is unbounded in general. Yang, Dwyer, and Rothermel [5] apply model checking incrementally to successive program versions. Beyer and Wendler [6] study conditional model checking, precision reuse, and verification witnesses as ways to reuse earlier verification runs; Beyer et al. [21] evaluate precision reuse on 1,119 revisions of 62 Linux device drivers. Lauterburg et al. [20] make state-space exploration, which they describe as the essence of explicit-state model checking, incremental; on 24 program evolutions and exploration changes, their abstract reports speedups in 22 cases, with a median of 42%. Kähler and Wilke [19] study the dynamic complexity of LTL model checking. These works take the change at the level of the state graph or the program. We study changes at the level of a distributed protocol specification.

Reuse of inductive invariants after a change. Chockler et al. [22] save the inductive invariant found by IC3 for one hardware design and, after a revision, keep the largest subset of its clauses that is still inductive. On HWMCC designs with random mutations of about 1% of the assignments, they report overall speedups of 3.04 (original re-checked after the mutant) and 1.35 (mutant after the original); between two versions of an IBM design that differ only in the driver, 3.13 and 2.22. The factors of about 30 and 61 measure re-checking an unchanged model. Their method also covers property changes in principle, but the experiments vary only the design. The SAT calls of their reuse step are bounded by the number of discarded clauses; the total cost is not bounded. IPDR [23] carries PDR lemmas over between instances whose constraints are tightened or relaxed; its instances are parameter variations rather than code revisions. Both reuse results at the level of the transition relation of a hardware or program model. Neither studies distributed protocols or bounds the number of verifier calls as a function of a specification edit.

Proof maintenance. Woos et al. [24] verify Raft in a proof assistant with 90 invariants and report proof maintenance as the main difficulty, since proving one invariant often required changing others. They propose a methodology to make proofs robust to change; they do not measure or bound its cost.

Neuro-symbolic invariant inference. IC3Syn [2] and I3DP [1] infer inductive invariants for 29 distributed protocols, including MongoLoglessDynamicRaft. Judging from the abstracts of [2] and [1], neither addresses re-verification after protocol change. Our measurements concern the kind of re-checking such inference loops repeatedly perform. In I4 [7], "incremental" means generalizing a finite-instance invariant to an unbounded one; it is unrelated to protocol evolution.

Other uses of "incremental". Frenkel, McMillan, Padon, and Shoham [8] decompose a safety proof into a sequence of simpler steps, not adapting a proof when the system is modified. Under natural restrictions, each of their proof rules strictly increases proof power, a useful model for rule-based separations.

Related tools and benchmarks. ProGS [9] performs LLM-based formal model synthesis guided by proof sketches, represented as trees; verification failures are localized to nodes or subtrees of the sketch, which enables targeted repair. ProGS constructs a model, whereas we measure re-verification after a change to an existing specification; such localization is the kind of structure a reuse analysis would need. Agentic Model Checking [10] performs compositional bounded model checking in which the cost of each query scales with the state space of a single function. TLA+-Bench [11] is an execution-grounded benchmark whose task is to produce a TLA+ module from a natural-language description of a system; its task does not start from an existing specification.

8 Threats to validity

Two commits, one repository, one model checker: this is a pilot study, not a law. Each result is a single observation, and we performed no statistical test. Result 1 times are means of three runs; Result 2 times come from two runs in which parent and child were checked concurrently; the 4-server time is a single run. State-overlap figures are computed up to server symmetry (Section 5) and are not measurements of transition reuse; we did not measure Δ directly. Other limits: the two commits were chosen by hand, which may bias the selection; all runs use one TLC worker and the default heap, and timings may change with more workers or memory; Result 2 uses 3 servers, not the shipped 4, and the effect of the edit may depend on instance size; RaftMongo and MongoReplReconfig may not represent other protocols. The canonical comparison assumes that `-dump` writes, under `SYMMETRY`, one representative per orbit; the equality of our counts with TLC's distinct counts is consistent with this, but we did not check it in TLC's source. We measure cost only as exhaustive TLC time and state counts, not with Apalache, TLAPS or simulation. All runs were made by a single experimenter and have not been replicated by others.

Mitigations.

- Machine-checked integrity of every pair, under the per-change-type rules of Section 2.
- State counts verified identical across independent reruns.
- TLC reports, per run, its estimate of the probability that some reachable states were not checked because two distinct states shared a fingerprint (verbatim from the tool output): for the parent run ($n = 3{,}680{,}146$), 1.2×10^{-5} (optimistic calculation) and 3.3×10^{-7} (based on the actual fingerprint table); for the child run ($n = 2{,}413{,}165$), 5.1×10^{-6} and 3.6×10^{-7}. The risk of a missed state grows with the number of generated states, not only distinct ones, so we take the larger figure, about 10^{-5}, as the relevant one. It is small.
- Our intersection hashes canonical state serializations with MD5 (128 bits, about 6.1 million states per comparison, birthday value about 5×10^{-26}).

- Raw logs, analysis scripts and the refinement check are available at https://github.com/amayene567/FormalSpecToolkit; the specifications are referenced by commit hash.

9 Acknowledgments, licensing and data

The specifications studied are the work of their MongoDB authors, in particular the authors of the studied commits: A. Jesse Jiryu Davis (RaftMongo, commit 348880238d) and Siyuan Zhou (MongoReplReconfig, commit dd27888039). At both commits, the headers of RaftMongo.tla and MongoReplReconfig.tla state "Copyright 2019 MongoDB, Inc." and a license under the Creative Commons Attribution 3.0 United States License (http://creativecommons.org/licenses/by/3.0/us/), added in commit bc754e818c (SERVER-45070), an ancestor of both. This report does not redistribute the specifications; they are referenced by commit hash [12] and can be retrieved with git show at those commits, or regenerated with pilot/fetch_specs.sh in our repository. MongoDB's third-party notices state that some of its TLA+ specifications are based on the Raft TLA+ specification by Diego Ongaro.

References

[1] W. Cao, G. Wu, Y. Yao, H. Wei, T. Chen, X. Ma. I3DP: Neuro-symbolic inductive invariant inference for distributed protocols. In *SOSP '26: Proceedings of the ACM SIGOPS 32nd Symposium on Operating Systems Principles*, pp. 620–635, 2026. https://doi.org/10.1145/3830418.3843876

[2] W. Cao, G. Wu, Y. Yao, H. Wei, T. Chen, X. Ma. Synthesizing inductive invariants for distributed protocols via IC3 and large language models. arXiv:2605.24619, 2026. https://arxiv.org/abs/2605.24619

[3] W. Schultz. Towards autonomous protocol proofs. Personal blog, 2026-04-03. https://will62794.github.io/formal-methods/2026/04/03/autonomous-protocol-proofs.html

[4] O. Sokolsky, S. A. Smolka. Incremental model checking in the modal mu-calculus. CAV 1994, LNCS 818, pp. 351–363. https://doi.org/10.1007/3-540-58179-0_67

[5] G. Yang, M. B. Dwyer, G. Rothermel. Regression model checking. ICSM 2009, pp. 115–124. https://doi.org/10.1109/ICSM.2009.5306334

[6] D. Beyer, P. Wendler. Reuse of verification results: Conditional model checking, precision reuse, and verification witnesses. SPIN 2013, LNCS 7976, pp. 1–17. https://www.sosy-lab.org/research/pub/2013-SPIN.Reuse_of_Verification_Results.pdf

[7] H. Ma, A. Goel, J.-B. Jeannin, M. Kapritsos, B. Kasikci, K. A. Sakallah. I4: Incremental inference of inductive invariants for verification of distributed protocols. SOSP 2019. https://web.eecs.umich.edu/~manosk/assets/papers/i4-sosp19.pdf

[8] E. Frenkel, K. L. McMillan, O. Padon, S. Shoham. Simplifying safety proofs with forward-backward reasoning and prophecy. *Proc. ACM Program. Lang.* 10(PLDI), Article 263, June 2026. https://doi.org/10.1145/3808341

[9] H. Wang, X. Zuo, Y. Cai, N. K. Singh, Y. Ait Ameur, J. S. Dong. Formal model construction guided by model-based proof sketches. arXiv:2609.29870, 2026. https://arxiv.org/abs/2609.29870

[10] Y. Sun, J. Liu, D. Kroening, J. Xue. Agentic model checking. arXiv:2605.21434, 2026. https://arxiv.org/abs/2605.21434

[11] A. Bisharat, E. Spencer, B. Ortiz, et al. TLA+-Bench: An execution-grounded benchmark and dataset for natural-language to TLA+ specification generation. arXiv:2607.23425, 2026. https://arxiv.org/abs/2607.23425

[12] MongoDB commits (mongodb/mongo): 348880238de3229826053491b1163c88dc3a0575 (SERVER-45853); dd278880390a0dda81ef7e9866be4c16b7571b4a (SERVER-47430, cherry-pick of 6e182b5c907f2965516b852311c9be6da05932d8); bc754e818ce974352bd76c850ec8dbc36620e765 (SERVER-45070). https://github.com/mongodb/mongo

[13] MongoDB Jira: SERVER-57262, "Allow nodes to vote for candidates with higher config" (closed, fixed in 4.4.9, 5.0.3, 5.1.0-rc0); SERVER-58788, "Update TLA+ to allow voting for candidate with higher config" (open). https://jira.mongodb.org/browse/SERVER-57262, https://jira.mongodb.org/browse/SERVER-58788 (accessed 2026-10-29)

[14] TLA+ tools v1.7.4 (tag v1.7.4 = commit 5a47802b5c391f59ecdd44117981f4ff8c0656ba); tla2tools.jar reports "TLC2 Version 2.19 of 08 August 2024 (rev: 5a47802)". The official release jar of v1.7.4, which reports this same version string, has SHA-256 936a262061c914694dfd669a543be24573c45d5aa0ff20a8b96b23d01e050e88. https://github.com/tlaplus/tlaplus/releases

[15] TLA+ Proof System documentation, "Running TLAPS". https://nightly.tlapl.us/doc/prover/runningTlaps.html (accessed 2026-10-29)

[16] K. R. M. Leino, V. Wüstholz. Fine-grained caching of verification results. CAV 2015, LNCS 9206, pp. 380–397. https://doi.org/10.1007/978-3-319-21690-4_22

[17] W. Schultz, E. Ashton, H. Howard, S. Tripakis. Interactive safety verification of distributed protocols by inductive proof decomposition. NFM 2026, LNCS 16622, pp. 3–24. https://doi.org/10.1007/978-3-032-28079-4_1

[18] G. M. Swamy, R. K. Brayton, V. Singhal. Incremental methods for FSM traversal. ICCD 1995, p. 590.

[19] D. Kähler, T. Wilke. Program complexity of dynamic LTL model checking. CSL 2003, LNCS 2803, pp. 271–284. https://doi.org/10.1007/978-3-540-45220-1_23

[20] S. Lauterburg, A. Sobeih, D. Marinov, M. Viswanathan. Incremental state-space exploration for programs with dynamically allocated data. ICSE 2008, pp. 291–300. https://doi.org/10.1145/1368088.1368128

[21] D. Beyer, S. Löwe, E. Novikov, A. Stahlbauer, P. Wendler. Precision reuse for efficient regression verification. ESEC/FSE 2013, pp. 389–399. https://doi.org/10.1145/2491411.2491429

[22] H. Chockler, A. Ivrii, A. Matsliah, S. Moran, Z. Nevo. Incremental formal verification of hardware. FMCAD 2011, pp. 135–143. https://www.cs.utexas.edu/~hunt/FMCAD/fmcad11/papers/8.pdf

[23] M. Blankestijn, A. Laarman. Incremental property directed reachability. ICFEM 2023, LNCS 14308, pp. 208–227. https://doi.org/10.1007/978-981-99-7584-6_13

[24] D. Woos, J. R. Wilcox, S. Anton, Z. Tatlock, M. D. Ernst, T. Anderson. Planning for change in a formal verification of the Raft consensus protocol. CPP 2016, pp. 154–165. https://homes.cs.washington.edu/~mernst/pubs/raft-proof-cpp2016.pdf